

TarAIntino: Application Design Document

Solution Architecture & Technical Architecture

Mathilde Cros, Robert Debbas, Maddy Jin, Karlo Vrancic

{mathildecros, robertdebbas, wenyijin, karlovrancic}@g.harvard.edu

Abstract

This document describes the application architecture of TarAIntino, an end-to-end machine learning pipeline that generates personalized movie trailers based on user preferences. The system employs a microservices architecture with six containerized services orchestrated via Docker Compose. The pipeline ingests user preferences through a Bayesian adaptive quiz, generates movie concepts via LLM-powered agents, decomposes narratives into trailer scenes, and produces video content using state-of-the-art generative AI APIs. This document details the solution architecture, technical stack, data flow, and code organization.

1. Solution Architecture

1.1 System Overview

TarAIntino is a production-grade MLOps system that transforms abstract user preferences into personalized AI-generated movie trailers. The system addresses a critical infrastructure gap: while generative models advance rapidly, the orchestration layer connecting user intent to coherent visual output remains underdeveloped.

The architecture follows a **microservices pattern** with clear service boundaries, enabling independent scaling, fault isolation, and technology heterogeneity. Services communicate via RESTful HTTP APIs over a private Docker bridge network, with the Next.js frontend serving as the API gateway and orchestration layer.

1.2 Microservices Architecture

The system comprises six core services, each deployed as an independent Docker container:

Service	Port	Responsibility
ChromaDB	8000	Vector database storing MovieLens Tag Genome embeddings for similarity search
Quiz Service	8082	Bayesian preference elicitation and recommendation engine
Screenplay Writer	8080	LLM-based movie concept generation from reference films
Scene Decomposer	8001	Trailer scene breakdown with character design prompts
Video Generator	8003	Image and video generation via Google Gemini and VEO 3.1
Frontend	3000	React-based UI and API gateway orchestrating the pipeline

All services are orchestrated via `docker-compose.yml` with health checks, dependency ordering, and automatic restart policies. The `chroma-init` service populates the vector database on

first startup from Google Cloud Storage.

1.3 Data Flow Pipeline

The end-to-end pipeline executes as a sequential workflow, orchestrated by the frontend's server-side API route:

Pipeline Stages:

- Quiz Phase:** User answers 5 questions rating movie tags (1–10 scale). The Quiz Service updates a Bayesian posterior to compute a taste vector.
- Recommendation:** The taste vector queries ChromaDB for k-nearest neighbor movies using cosine similarity on Tag Genome embeddings.
- Movie Generation:** Top-5 movie titles are sent to Screenplay Writer, which uses an LLM to generate an original movie concept.
- Scene Decomposition:** The movie concept is decomposed into trailer scenes with character designs and video prompts.
- Video Production:** Character reference images are generated via Gemini, scene videos via VEO 3.1, then stitched into a trailer.
- Delivery:** Final trailer is uploaded to GCS; frontend polls for completion and displays the video.

1.4 API Communication

Services communicate via JSON over HTTP. The frontend acts as the API gateway, calling services sequentially:

Service	Endpoint	Purpose
Quiz	POST /quiz/start	Initialize session
Quiz	POST /quiz/answer	Submit rating
Quiz	POST /quiz/complete	Get taste vector
Quiz	POST /recommend	Get recommendations
Screenplay	POST /generate-movie	Generate concept
Decomposer	POST /generate-trailer	Scene breakdown
Video	POST /generate/trailer	Produce video

All services expose GET /health endpoints for Docker health checks and monitoring.

2. Technical Architecture

2.1 Backend Stack

All backend services are implemented in **Python 3.11** using the **FastAPI** framework with **Uvicorn** ASGI server.

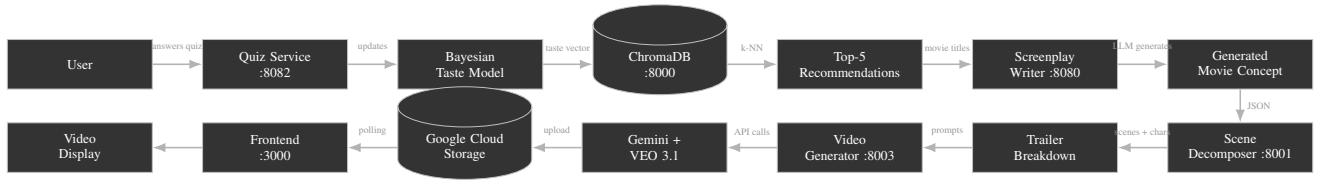


Figure 1: End-to-end data flow: (Top) Preference elicitation and movie concept generation. (Bottom) Scene decomposition and video production. The frontend orchestrates all service calls via its API gateway.

Core Dependencies:

- **FastAPI 0.115+:** High-performance async web framework with automatic OpenAPI documentation
- **Pydantic 2.9+:** Data validation and serialization via Python type hints
- **Uvicorn:** Lightning-fast ASGI server
- **httpx:** Async HTTP client for service-to-service calls
- **Service-Specific Dependencies:**
 - **Quiz Service:** NumPy 1.26, SciPy 1.13, scikit-learn 1.5, ChromaDB client, Pandas 2.2
 - **Screenplay Writer:** OpenAI SDK (OpenRouter compatibility), requests
 - **Scene Decomposer:** OpenAI SDK, python-dotenv
 - **Video Generator:** google-genai 1.52+, google-cloud-aiplatform 1.71+, google-cloud-storage 2.19+, vertexai

2.2 Frontend Stack

The frontend is a modern React application built with **Next.js 15.5.4** and **React 19.1.0**, using TypeScript for type safety.

Core Technologies:

- **Next.js 15:** React framework with App Router, server components, and API routes
- **React 19:** Latest React with concurrent features
- **TypeScript 5:** Static typing for reliability
- **Tailwind CSS 4:** Utility-first CSS framework
- **Animation & UI Libraries:**
 - **Framer Motion 12.23:** Declarative animations
 - **GSAP 3.13:** High-performance animation library
 - **Lottie:** Vector animations via @lottiefles/react-lottie-player
 - **Radix UI:** Accessible component primitives
 - **Lucide React:** Icon library

The frontend uses Next.js API routes as a BFF (Backend-for-Frontend) pattern, orchestrating calls to backend microservices.

2.3 Machine Learning Components

2.3.1 Bayesian Taste Model

The Quiz Service implements `FullCovarianceTasteModel`, a Bayesian inference model for preference elicitation:

- **Prior:** Full-covariance Gaussian $\mathcal{N}(\mu_0, \Sigma_0)$ computed from MovieLens Tag Genome statistics

- **Posterior Update:** Kalman filter-like update equations:

$$\mathbf{k} = \Sigma_{:,j} / (\Sigma_{jj} + \sigma^2)$$

$$\hat{\theta} \leftarrow \hat{\theta} + \mathbf{k}(y - \hat{\theta}_j)$$

$$\Sigma \leftarrow \Sigma - \mathbf{k}\Sigma_{j,:}^T$$

- **Question Selection:** Maximum variance heuristic—select tag $j^* = \arg \max_j \Sigma_{jj}$ among unasked questions
- **Dimensionality:** $D \approx 1128$ (Tag Genome embedding dimension)
- **Rating Scale:** User ratings 1–10 normalized to $[0, 1]$ probability

2.3.2 Vector Embeddings

ChromaDB stores movie embeddings derived from the MovieLens Tag Genome:

- **Collection:** `movie_tag_relevance_cos`
- **Distance Metric:** Cosine similarity
- **Data Source:** Tag relevance scores (0.0–1.0) for 1,129 tags across 62,000 films
- **Ingestion:** Automated via `chroma-init` service from GCS bucket

2.4 External Service Integrations

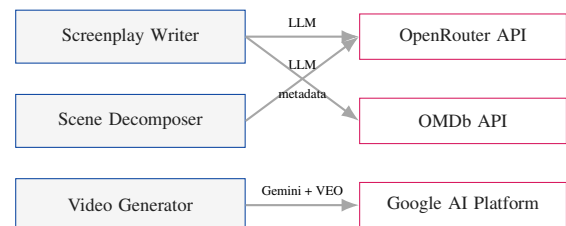


Figure 2: External API integrations by service.

OpenRouter API:

- Unified LLM gateway supporting multiple providers
- Screenplay Writer: `google/gemini-3-pro-preview`
- Scene Decomposer: `google/gemini-3-pro-preview`

OMDb API:

- Open Movie Database for fetching movie metadata
- Used by Screenplay Writer for reference film context

Google AI Platform:

- **Gemini API:** Character reference image generation
- **VEO 3.1:** State-of-the-art video generation from prompts
- **Cloud Storage:** Video asset storage (bucket: `tarantino-output`)

2.5 Infrastructure & DevOps

2.5.1 Containerization

All services are containerized using Docker with multi-stage builds:

- **Backend** **Base** **Image:**
python:3.11-slim-bookworm
- **Frontend Base Image:** node:20-alpine (multi-stage)
- **Package Manager:** uv for fast Python dependency installation
- **Network:** Custom bridge network
tarantino-network
- **Volumes:** Persistent chroma_data volume for vector database

2.5.2 CI/CD Pipeline

GitHub Actions workflows automate quality checks:

- **ci.yml:** Main pipeline—linting, type-checking, Docker-based tests
- **backend-ci.yml:** Python-specific checks (Black, Ruff, Pytest)
- **frontend-ci.yml:** TypeScript, ESLint, Next.js build verification

2.5.3 Code Quality Tools

- **Black:** Python code formatter (line-length: 100)
- **Ruff:** Fast Python linter (pycodestyle, pyflakes, isort, flake8-bugbear)
- **Pytest:** Testing framework with coverage reporting
- **ESLint:** TypeScript/JavaScript linter
- **Husky:** Git pre-commit hooks for automatic formatting

3. Code Organization

3.1 Directory Structure

The project follows a monorepo structure with service isolation:

```
AC215-TarAIntino-Big/
|-- .github/workflows/      CI/CD pipelines
|-- src/
| |-- quiz-vector/          Quiz + RAG service
| |-- screenplay-writer/    Movie generation
| |-- scene-decomposer/     Trailer breakdown
| |-- video-generator/      Video production
| |-- frontend/             Next.js application
|-- tests/                  Integration tests
|-- docker-compose.yml      Orchestration
|-- pyproject.toml          Python config
|-- Makefile                Dev automation
```

3.2 Service Boundaries

Each service maintains strict boundaries with its own:

- `pyproject.toml`: Service-specific dependencies
- `Dockerfile`: Container definition
- `src/`: Application code

- `tests/`: Unit tests

- `.env.example`: Environment configuration template

Key Implementation Files:

- Bayesian model
src/quiz-vector/src/quiz_service/model.py
- API gateway:
src/frontend/app/api/generate-video/route.ts
- Video generation:
src/video-generator/generate.py
- LLM screenplay:
src/screenplay-writer/src/movie-pipeline/movie.py